

Skills for Agentic Coding Teams

Helping people and teams grow with AI

Free download:

<https://connexxo.com/resources/agentic-coding-teams/>

The book in six parts



- I The Landscape
- II Teams and People
- III Practices and Standards
- IV The Hidden Costs
- V Roles: Scrum Masters and Agile Coaches
- VI Governance, Sovereignty, and Reality

Sources and method

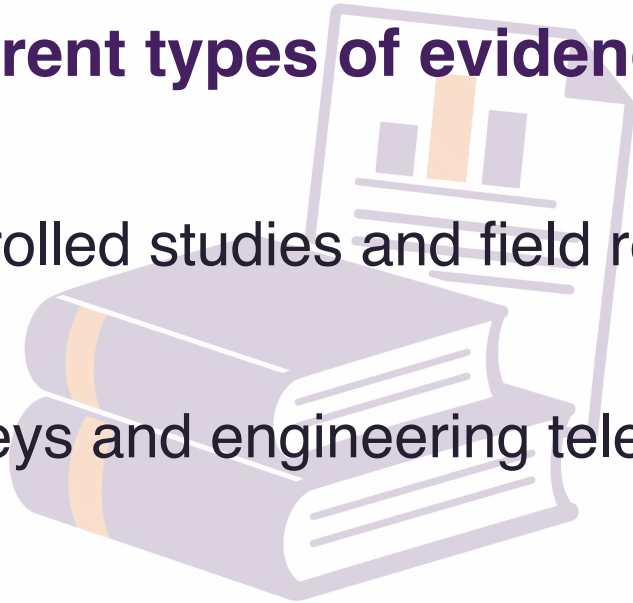
Different types of evidence

Controlled studies and field research

Surveys and engineering telemetry

Reports and ideas from practitioners

Removed “untrustworthy sources”

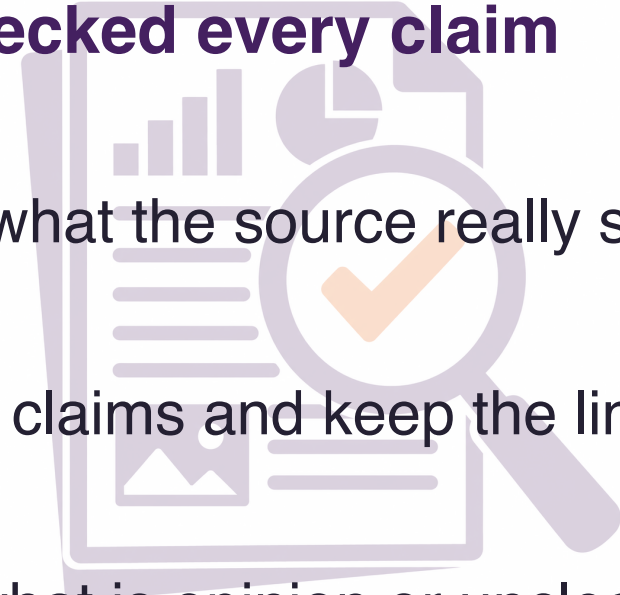


We checked every claim

Check what the source really says

Correct claims and keep the limits

Show what is opinion or unclear



**AI is an amplifier.
Good and bad get bigger.**

The structure of your organisation makes the difference.

What AI makes bigger

Strengths

Shared standards

Collaboration

Careful reviews

Weaknesses

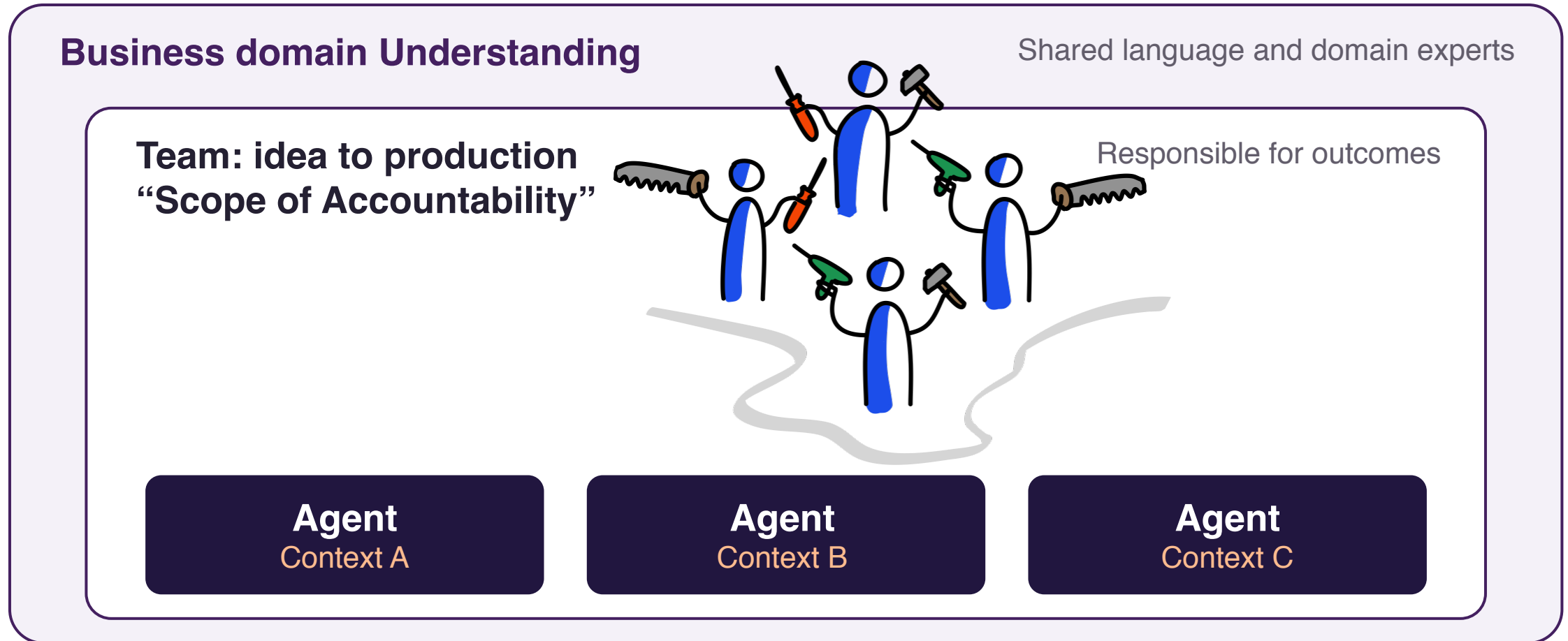
Missing standards

Isolated teams / Silos

Rushed work

An agent makes good and bad standards bigger.

What a team must own



Limit what the agent may do. Do not limit what the team owns.

Skills developers must have

Learning to judge work done with agents

5. Skills Developers Must Have

The skills picture has shifted markedly. The skills moving up the value curve, drawn from the Stack Overflow 2025 Developer Survey [77] and the Anthropic findings [7], are architectural thinking, code review at speed and scale, problem decomposition into agent-shaped chunks, specification and prompting, context engineering, verification and validation, and domain expertise. Reading code is becoming more valuable than writing it, while syntax memorisation, boilerplate, routine bug fixing in well-tested codebases, and simple integration work are being commoditised.¹

A deeper problem looms behind this shift in the skill set: how to train the next generation of senior engineers once the apprenticeship work has been automated. That is the subject of the chapter on growing juniors into seniors.

A handful of skills matter more than ever: critical thinking, which research suggests can atrophy under heavy AI use; clear communication and writing, because clear specifications become clear prompts become clear systems; security awareness, since every developer is now part of the security boundary; operational thinking for non-deterministic systems; asynchronous coordination of multiple parallel agent sessions; and cognitive-load management, which a later chapter treats in detail. A useful way to picture the shift is that the developer is moving from being the forward who scores to being the playmaker who creates the conditions for scoring: different skills, equally demanding, but the value of pure execution has dropped.

¹This framing is this book's interpretation. It is broadly supported by the Stack Overflow 2025 survey, which finds the bottleneck shifting "from writing code to verifying AI-generated code", but the exact phrasing is not a direct finding of any single cited study.

Skills ~~developers~~ people must have

Learning to judge work done with agents

5. Skills Developers Must Have

The skills picture has shifted markedly. The skills moving up the value curve, drawn from the Stack Overflow 2025 Developer Survey [77] and the Anthropic findings [7], are architectural thinking, code review at speed and scale, problem decomposition into agent-shaped chunks, specification and prompting, context engineering, verification and validation, and domain expertise. Reading code is becoming more valuable than writing it, while syntax memorisation, boilerplate, routine bug fixing in well-tested codebases, and simple integration work are being commoditised.¹

A deeper problem looms behind this shift in the skill set: how to train the next generation of senior engineers once the apprenticeship work has been automated. That is the subject of the chapter on growing juniors into seniors.

A handful of skills matter more than ever: critical thinking, which research suggests can atrophy under heavy AI use; clear communication and writing, because clear specifications become clear prompts become clear systems; security awareness, since every developer is now part of the security boundary; operational thinking for non-deterministic systems; asynchronous coordination of multiple parallel agent sessions; and cognitive-load management, which a later chapter treats in detail. A useful way to picture the shift is that the developer is moving from being the forward who scores to being the playmaker who creates the conditions for scoring: different skills, equally demanding, but the value of pure execution has dropped.

¹This framing is this book's interpretation. It is broadly supported by the Stack Overflow 2025 survey, which finds the bottleneck shifting "from writing code to verifying AI-generated code", but the exact phrasing is not a direct finding of any single cited study.

What becomes more important

More important

Architecture and domain expertise

Specification and context

Review and verification

Less important

Syntax and boilerplate

Routine bug fixing

Simple integration work



Less writing code, more guiding the work.

Judgement and clear language

Critical thinking

Question assumptions

Look for proof that the answer is wrong

Know what is still unclear

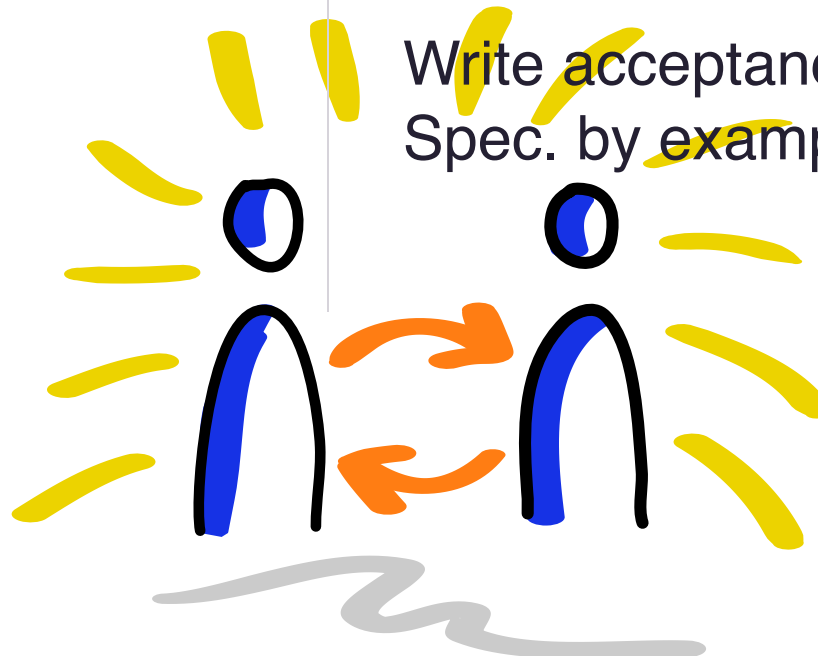
A team can only check what it understands.

Clear communication and writing

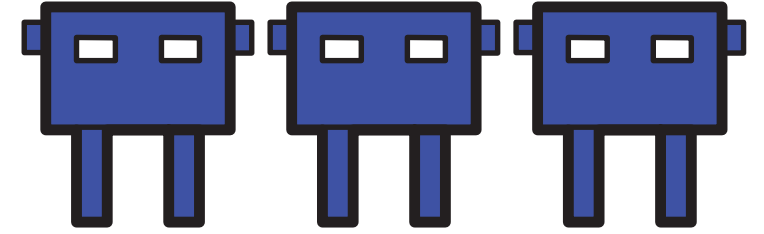
Say the goal and the limits

Ubiquitous Language

Write acceptance criteria you can check:
Spec. by example, BDD, ...



Specification and context



Domain language

Acceptance criteria in business terms

Convention files

No unwritten rules, be explicit!

Small tasks for agents

One bounded context at a time

An agent does not ask. It guesses one meaning.

Verification and review

Tests first

Tests tell the agent what to build

Test the tests

Mutation testing finds weak tests

Review the purpose

Small PRs, check the architecture

Without tests, technical debt grows very fast.

My experience: reviewing is harder

Writing it myself

I know why each line is there

I build understanding as I go

I stop when I am tired

Reviewing LLM output

I must rebuild the reasoning

Plausible code/text hides mistakes

The next proposal is ready

Understanding code you did not write is a burden.

Security and operations

Security awareness

Know the trust boundaries

Check that packages actually exist

Know where code and data go

Non-deterministic systems

Compare results across many runs

Trace actions and find failures

Set limits and plan recovery

You are still responsible after the agent answers.

Parallel work and limited attention

Asynchronous coordination

Split work with clear boundaries

Show dependencies and handoffs

Plan checkpoints and integration

Managing cognitive load

Limit work in progress

Keep time to understand

Share the review work

Parallel work needs time to coordinate and to review.

From Junior to Senior

How do I get the necessary scars?

Becoming Senior: How judgement develops

Guided responsibility

Seniors and juniors work with agents together

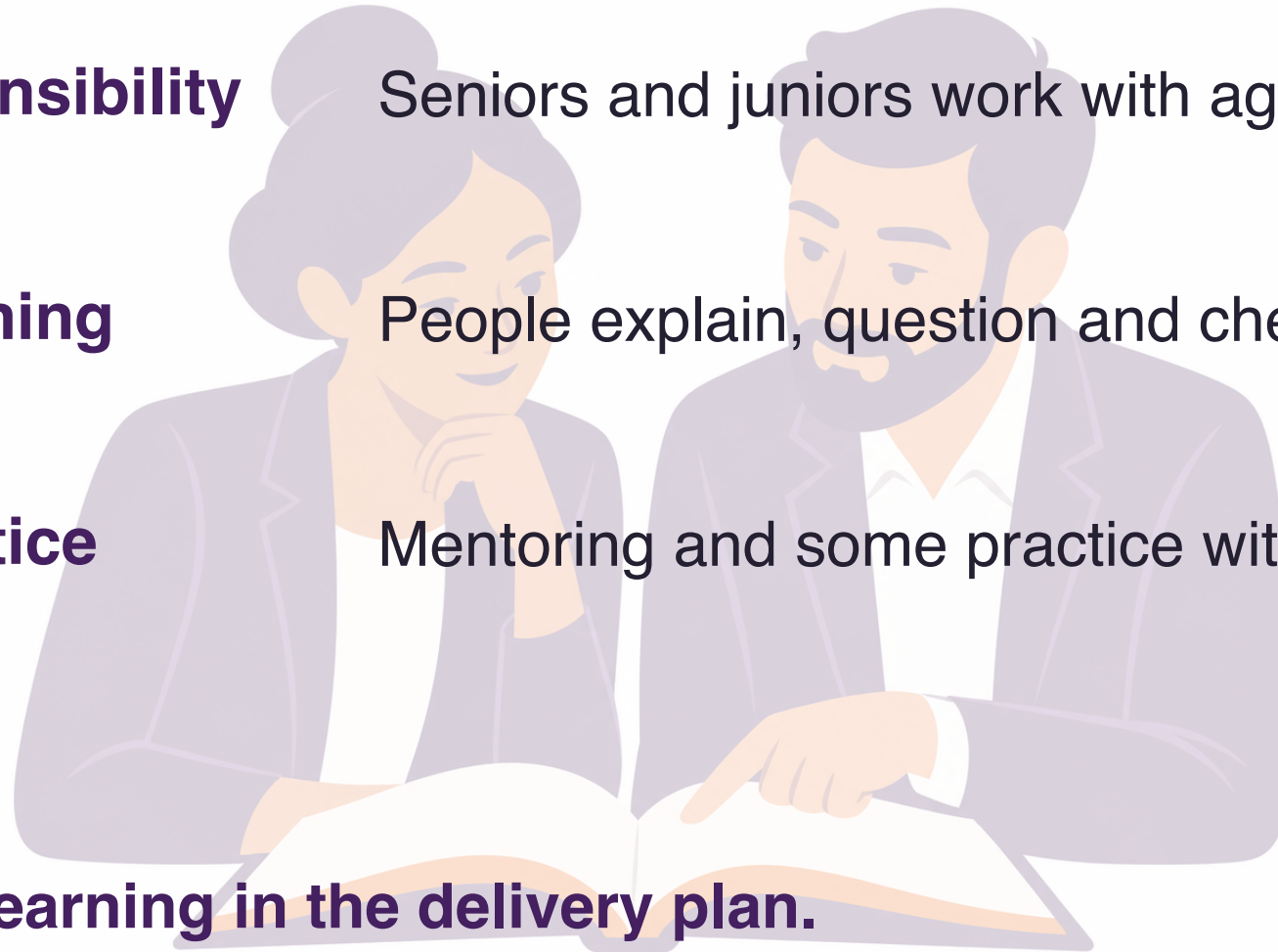
Visible reasoning

People explain, question and check decisions

Planned practice

Mentoring and some practice without AI

Plan time for learning in the delivery plan.



How juniors learn to judge

Preceptor model

A senior watches a junior check AI output

Ask, do not delegate

Use AI to understand concepts

Time without AI

No-AI work blocks and focus time

The juniors of today are the seniors of 2032.

Business analysts

What changes

The analyst works inside the team

Agents guess, they do not ask

Own the shared domain words

What to learn

Domain modelling and Event Storming

Acceptance criteria in business words

Enough architecture to read the code

Keep the glossary in the codebase, not in a wiki.

Other Roles...

Culture Follows Structure
- C. Larman

Product owners

What changes

AI writes first drafts of user stories

Less manual backlog work

Responsible for outcomes

The tool proposes. The product owner decides.

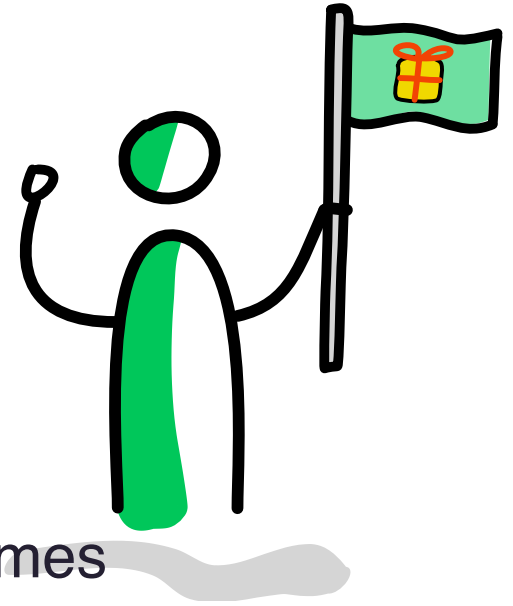
What to learn

Describe work as outcomes

Question priorities suggested by AI

See if a backlog is good, not just tidy

Understand where VALUE is!



Scrum Masters and coaches

What changes

AI does the admin work (that was never a SM job anyway!)

Facilitation and coaching stay

Teams now include agents

What to learn

Understand what agents may do alone

Coach the right level of trust

Look at team speed, not personal speed



Less organising, more judging and caring for the team.

Leadership

What changes

Personal speed can hide team costs

Juniors are still needed

Agents need clear rules

Never judge people with agent data.

What to learn

Measure team flow, not output

One person owns the token budget

Decide where code and data go



**Faster coding moves
the waiting to other places.**

Look for bottlenecks in the whole system

Meaning and context

Shared goal

Agreed domain words and acceptance criteria

Useful context

The right information and reliable data

Written standards

Conventions, architecture and design systems



Unclear requirements create rework, very fast.

Verification and delivery

Verification capacity

Purpose, correctness, security and review

Integration capacity

Small changes, reliable tests and pipelines

Operational capacity

Observability, release and recovery



What counts: reliable delivery to users.

Decisions, collaboration and learning

Slow decisions

Open tradeoffs and unclear ownership

Coordination costs

Handoffs, different assumptions, lost context

Time to learn

Senior time and the path to expertise



Team results depend on shared understanding.

Governance and economics

What makes use safe

Who decides, who is responsible

Security, confidentiality and data protection

Code origin, licences and hosting

What keeps the value

Token costs and review costs

Infrastructure and people to run it

New workflows and time to learn

Success depends on decisions of the whole organisation.

Questions for the next steps

What must people learn to judge well?

Which team problem must we remove first?

How will we see that things are better?